

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División ciencias de la ingeniería
Organización de Lenguajes y Compiladores 2

Manual Técnico

Proyecto 1 — Contacto 3xtrat3rr3str3D

Estudiante: Brandon Gustavo Güinac Román
Carné: 201931217
Catedrático: Mario Ramirez

Introducción

El presente documento constituye la documentación técnica del Proyecto 1 del curso de Organización de Lenguajes y Compiladores 2, correspondiente al desarrollo de un conjunto de compiladores para tres lenguajes de entrada distintos —denominados *Y?*, *Zetariano* y *Pig Latin*— junto con un entorno de desarrollo integrado (IDE) construido para editar, resaltar y ejecutar programas escritos en dichos lenguajes. El propósito de este manual es describir, con el nivel de detalle que exige un entregable académico de esta naturaleza, las tecnologías empleadas, la arquitectura de clases del sistema, la especificación léxica y sintáctica de cada lenguaje, y las reglas de compatibilidad de tipos que rigen el análisis semántico.

Cada uno de los tres lenguajes cuenta con su propio analizador léxico y sintáctico generado a partir de una gramática independiente escrita en ANTLR4, y con su propio analizador semántico. Sin embargo, los tres comparten un mismo núcleo de estructuras de datos semánticas —tabla de símbolos, sistema de tipos y manejo de errores—, lo que permite, en particular, que un programa escrito en Pig Latin importe y utilice definiciones provenientes de archivos `.y` o `.z` sin que exista duplicación de lógica entre los tres analizadores. Esta decisión de diseño se refleja directamente en la organización de clases descrita en la sección 2.

Para quien desee consultar el razonamiento detrás de cada decisión de implementación —incluyendo los casos donde se detectaron y corrigieron errores durante el desarrollo de la interfaz gráfica— se recomienda revisar el archivo `README.md` ubicado en la raíz del repositorio, que documenta el proceso de construcción del proyecto con mayor extensión que la que corresponde a un manual técnico.

1. Tecnologías utilizadas

Java 17 es el lenguaje en el que está escrita la totalidad del proyecto, tanto el motor de compilación como la interfaz gráfica. La versión del lenguaje y del bytecode objetivo se fija explícitamente en el archivo `pom.xml` mediante las propiedades `maven.compiler.source` y `maven.compiler.target`.

Maven se utiliza como herramienta de construcción del proyecto: gestiona las dependencias, coordina la compilación y, mediante el complemento `antlr4-maven-plugin`, invoca automáticamente al generador de ANTLR4 antes de compilar el código Java, de modo que los analizadores léxicos y sintácticos quedan disponibles como código fuente generado dentro del ciclo de construcción normal del proyecto.

ANTLR4 (en su versión de tiempo de ejecución 4.13.1) es el generador de analizadores léxicos y sintácticos empleado para los tres lenguajes. Cada lenguaje cuenta con su propia gramática, ubicada en `src/main/antlr4/com/company/{y,zetariano,pig}`, a partir de la cual ANTLR4 produce el `lexer`, el `parser` y las clases de visitante correspondientes.

Java Swing (paquete `javax.swing`, incluido en el JDK estándar) es la base de toda la interfaz gráfica del proyecto —árbol de archivos, editor de texto, resaltado de sintaxis y consola de salida—, sin recurrir a ninguna librería externa de interfaz de usuario ni de resaltado de sintaxis. Esta restricción no es una preferencia de diseño sino un requisito explícito del enunciado del proyecto, que prohíbe el uso de librerías para el coloreado de código; por ello, el módulo de resaltado (descrito en la sección 2.3) está implementado íntegramente mediante expresiones regulares y las clases de estilizado de texto que provee el propio Swing.

JUnit 5 figura declarado como dependencia de alcance de prueba en `pom.xml`, pero la verificación funcional real del proyecto se realiza mediante una suite de comprobaciones propia, implementada en la clase `Main.java` y ejecutada junto con la construcción de Maven; el README documenta la razón de esta decisión.

C es el lenguaje de salida del compilador: la clase `GeneradorC` traduce la representación intermedia de cuartetos (código de tres direcciones) a código C, compilable con `gcc -Wall` sin advertencias, y sin depender de ninguna librería fuera de la biblioteca estándar del lenguaje (`stdio.h`, `stdlib.h`, `string.h`).

GitHub aloja el repositorio del proyecto, tal como lo exige el enunciado como parte de los entregables.

2. Arquitectura y diagrama de clases

Con el fin de que los diagramas puedan leerse sin saturación visual, la arquitectura del sistema se presenta agrupada en tres capas: el núcleo semántico compartido por los tres lenguajes, el conjunto de clases que realizan el análisis y la generación de código, y la interfaz gráfica. El proyecto contiene, además, un número adicional de clases de apoyo específicas de cada lenguaje —por ejemplo, un resolutor de tipos y un resolutor de acceso a miembros por cada uno de los tres analizadores— que se mencionan en el texto mas no se representan de forma individual en los diagramas, tanto por razones de espacio como porque su responsabilidad puede describirse adecuadamente en prosa.

2.1. Núcleo semántico

El paquete `com.company.semantico` concentra las estructuras de datos que los tres analizadores comparten sin duplicación: el sistema de tipos, la tabla de símbolos y el mecanismo de reporte de errores. La clase `Tipo` es inmutable y modela tanto los tipos primitivos de cada lenguaje como los tipos por referencia (estructuras, clases y arreglos); su método `esCompatibleCon(Tipo)` centraliza la regla de compatibilidad que se detalla en la sección 4 de este documento, de manera que dicha regla se evalúa en un único lugar del código en lugar de repetirse en cada analizador.

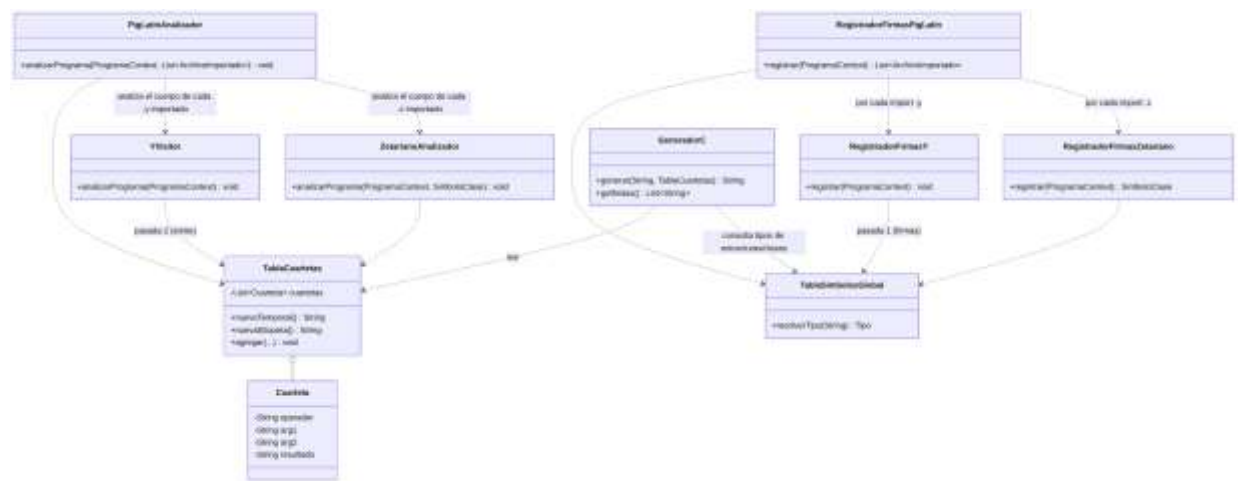
La clase `Simbolo` es la base común de toda entidad declarable —variables, campos, funciones, métodos— y de ella derivan `SimboloEstructura` (para los tipos compuestos de Y? y Pig Latin), `SimboloClase` (para las clases de Zetariano, con sus campos, métodos y constructores) y `SimboloInvocable` (para funciones y métodos, con su lista de parámetros, tipo de retorno y ámbito local propio). La resolución de nombres se apoya en `TablaSimbolos`, que forma una cadena de ámbitos mediante una referencia a su tabla padre, y en `TablaSimbolosGlobal`, que además indexa por nombre las estructuras, clases y funciones declaradas a nivel de programa y ofrece el método `resolverTipo(String)` para traducir un nombre de tipo textual a una instancia de `Tipo`.

El manejo de errores semánticos se realiza mediante `GestorErrores`, que acumula instancias de `ErrorSemantico` (mensaje, línea, columna y, opcionalmente, archivo de origen). El campo `archivo` de `ErrorSemantico` y el campo auxiliar `archivoActual` de `GestorErrores` fueron incorporados durante el desarrollo de la interfaz gráfica para resolver un problema real detectado en pruebas manuales: cuando un archivo `.pig` importa varios archivos `.y` o `.z`, los errores semánticos de todos los archivos se reportan en una sola lista consolidada, y sin una marca de procedencia era imposible distinguir a cuál de los archivos importados pertenecía cada error. Este mecanismo se describe con mayor detalle en la sección 6 del manual de usuario.

ninguna línea de C; `EmisorSentencias` (Pase B) recorre el mismo cuerpo una segunda vez, ya con los tipos resueltos, y emite la sentencia C real de cada cuarteta; y `GeneradorC` queda como orquestador: construye las tres clases anteriores, decide qué funciones, clases y estructuras son traducibles, y ensambla el archivo .c final. El método `getNotas()` de `GeneradorC` expone una única lista de notas que las tres clases colaboradoras reciben por referencia y alimentan en común, de modo que cualquier construcción del lenguaje de origen sin traducción a C se reporte en un solo lugar, sin interrumpir la generación del resto del programa.

En cuanto a arreglos: Y?, Zetariano y Pig Latin soportan arreglos de una dimensión de tipo entero, decimal o booleano, traducidos a C mediante `malloc` (arreglos dinámicos de Zetariano y Pig Latin, declarados con `new tipo[n]` o `series nombre[n] : tipo`) o mediante un arreglo fijo real de C (arreglos de tamaño literal de Y?, declarados directamente como `tipo nombre[N];`). La limitación conocida que permanece, a la fecha de este documento, es el arreglo cuyo tipo de elemento es una estructura o una clase (por ejemplo, `series personas[3] : Persona` en Pig Latin); `GeneradorC` detecta este caso y lo excluye de forma controlada —reportándolo mediante una nota— en lugar de generar código C inválido.

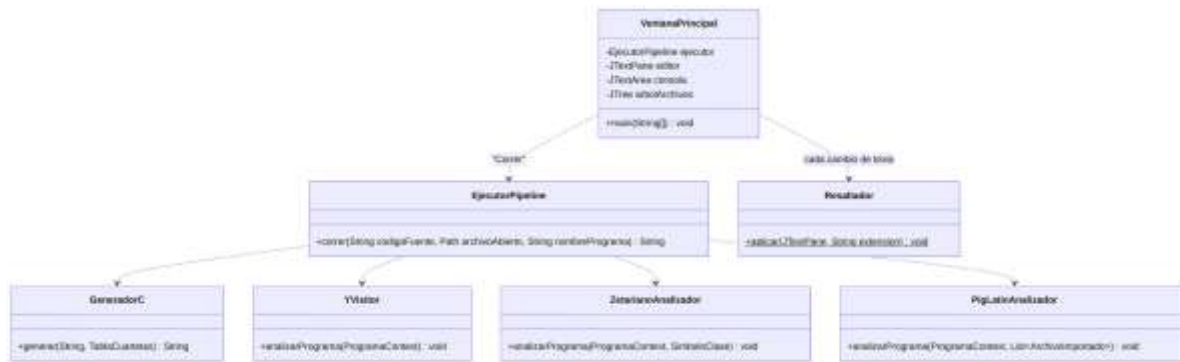
Tanto `ZetarianoAnalizador` como `PigLatinAnalizador` delegan a su vez la resolución de tipos y de acceso a miembros (`obj.campo`, `obj.metodo(...)`, `arreglo[i]`) en clases auxiliares propias de cada lenguaje (`ResolvedorTipoZetariano/ResolvedorAccesoZetariano` y `ResolvedorTipoPigLatin/ResolvedorAccesoPigLatin`), de modo que ninguno de los dos analizadores concentre por sí solo toda la lógica de resolución; estas clases no se representan en el diagrama por razones de espacio, pero se documentan con detalle en el README, en la sección dedicada a cada visitante.



2.3. Interfaz gráfica (IDE)

La interfaz gráfica reside en el paquete `com.company.gui` y se organiza alrededor de tres clases. `VentanaPrincipal` es el punto de entrada de la aplicación y aloja el árbol de archivos del proyecto,

el editor de código y la consola de salida; delega en `EjecutorPipeline` la ejecución del botón «Correr», que a su vez invoca al analizador correspondiente a la extensión del archivo abierto y, si el análisis concluye sin errores, a `GeneradorC` para producir el código C final. La tercera clase, `Resaltador`, se invoca cada vez que el contenido del editor cambia y aplica el coloreado de sintaxis descrito en la sección 3, sin depender de ninguna librería externa.



3. Palabras reservadas, símbolos y gramática

La fuente autoritativa de la gramática de cada lenguaje son los propios archivos ANTLR4 ubicados en `src/main/antlr4/com/company/{y,zetariano,pig}/*.g4`; lo que sigue es un resumen orientado a consulta rápida y no sustituye a dichos archivos.

3.1. Y? (extensión .y)

Y? es un lenguaje sensible a mayúsculas y minúsculas que emplea indentación significativa —de forma análoga a Python— para delimitar bloques de código, en lugar de recurrir a llaves. Las palabras reservadas del lenguaje son %estructuras, %funciones, estructura, definir, retornar, si, entonces, sino, contrario, elegir, caso, siempre, romper, continuar, para, mientras, hacer, verdadero y falso; sus tipos primitivos son entero, cadena, flotante, caracter y bool. Entre los símbolos y operadores reconocidos por la gramática se encuentran `->`, `&&`, `||`, `==`, `!=`, `++`, `--`, `<`, `>`, `!`, `+`, `-`, `*`, `/`, `=`, así como los signos de puntuación habituales (`:`, `;`, `,`, `.`, paréntesis, llaves y corchetes).

Un programa Y? (regla `programa`) consiste en una sección %estructuras opcional, con cero o más definiciones de estructura, seguida de una sección %funciones obligatoria, con una o más definiciones de función; el lenguaje no admite variables globales sueltas fuera de estas dos secciones. Cada sentencia (regla `sentencia`, que puede ser simple o compuesta) delimita el cuerpo de las construcciones `si`, `para`, `mientras`, `hacer`, `elegir` o el cuerpo de una función mediante tokens sintéticos de indentación y desindentación, generados por la clase `YLexerBase` a partir del token de nueva línea.

3.2. Zetariano (extensión .z)

Zetariano es un lenguaje orientado a objetos de estilo similar al de Java. Cada archivo `.z` define exactamente una clase pública, y el nombre del archivo debe coincidir con el nombre de dicha clase. Sus palabras reservadas son `public`, `private`, `class`, `void`, `new`, `if`, `else`, `switch`, `case`, `default`, `for`, `while`, `do`, `break`, `continue`, `return`, `true`, `false`, `null` y `this`; sus tipos primitivos son `int`, `double`, `char`, `boolean` y `String`. Los símbolos y operadores incluyen `++`, `--`, `&&`, `||`, `==`, `!=`, `<=`, `>=`, `+=`, `-=`, `*=`, `<`, `>`, `!`, `+`, `-`, `*`, `/`, `%` y `=`, además de los signos de puntuación habituales.

Un programa Zetariano (regla `programa`) es una clase con cero o más miembros, donde cada miembro es un campo, un constructor o un método; el cuerpo de cualquier método, constructor o estructura de control es un bloque delimitado por llaves (regla `bloque`). El enunciado del proyecto establece dos precisiones importantes sobre este lenguaje: los objetos creados con `new` viven en el heap, lo que se traduce, en el código C generado, como una reserva de memoria mediante `calloc` (véase `GeneradorC`); y el proyecto no contempla encapsulamiento, herencia ni polimorfismo, características que quedan reservadas para el Proyecto 2 del curso.

3.3. Pig Latin (extensión .pig)

Pig Latin es el lenguaje de entrada del programa completo —identificado por el marcador `MAIOR>`— y también es sensible a mayúsculas y minúsculas. A diferencia de Y? y Zetariano, no define tipos propios: su única función es importar y orquestar definiciones provenientes de archivos `.y` y `.z` ya existentes. Sus palabras reservadas son `import`, `esto`, `series`, `si`, `aliter`, `dum`, `facere`, `per`, `perge`, `interrumpe`, `novus`, `finis` y `FINIS`; sus tipos primitivos son `numerus`, `textum`, `decimalis` y `littera`, mientras que los valores booleanos se expresan mediante los literales `verum` y `falsus`. El programa se organiza en dos marcadores de sección: `VARIABLES>`, opcional, para variables, arreglos o estructuras globales, y `MAIOR>`, obligatorio, para el cuerpo principal del programa.

Entre los símbolos reconocidos se incluyen `++`, `--`, `&&`, `||`, `==`, `!=`, `<=`, `>=`, `>>` (usado para imprimir), `<<` (usado para leer), `<`, `>`, `!`, `+`, `-`, `*`, `/`, `%` y `=`. Un programa (regla `programa`) consiste en cero o más importaciones (`import carpeta.Archivo.z o .y`), seguidas de una sección de variables opcional y de la sección principal obligatoria; cada importación se resuelve, en `RegistradorFirmasPigLatin`, contra un archivo real en disco ubicado en una ruta relativa a la carpeta del archivo `.pig` —la convención exacta de resolución de rutas se documenta en el README, en la sección dedicada al visitante de Pig Latin—.

4. Reglas de compatibilidad de tipos

La regla general de compatibilidad de tipos, compartida por los tres lenguajes e implementada en un único método (`Tipo.esCompatibleCon`, paquete `com.company.semantico`), establece que dos tipos son compatibles en alguno de tres casos: cuando son exactamente el mismo tipo; cuando el tipo destino es decimal (`double` / `decimalis`) y el tipo origen es entero (`int` / `numerus`), en cuyo caso ocurre una promoción implícita equivalente a la de Java o C; o cuando el tipo origen es el literal `null` de Zetariano y el tipo destino es un tipo por referencia (estructura, clase o arreglo). En ningún caso existe conversión automática entre un tipo numérico y un tipo de cadena, ni entre un tipo booleano y cualquier otro tipo; ambas conversiones requieren una acción explícita del programador o, en el caso particular de Zetariano, el operador de concatenación que se describe a continuación.

4.1. Operadores aritméticos

Y? y Pig Latin son estrictos en este punto: los operadores `+`, `-`, `*` y `/` solo se aceptan entre operandos de tipo entero o decimal —el mensaje real que produce el compilador ante una violación de esta regla es «'+' solo aplica entre valores numericos, no ...»—. Zetariano es la única excepción, y únicamente para el operador de suma, tal como se explica en la sección 4.2. Ningún operando de tipo booleano, carácter, estructura o clase participa en una operación aritmética en ninguno de los tres lenguajes.

Operando A	Operando B	Y?	Zetariano	Pig Latin	Tipo resultante
entero	entero	Sí	Sí	Sí	entero
entero	decimal	Sí	Sí	Sí	decimal (promoción)
decimal	decimal	Sí	Sí	Sí	decimal
cadena	cualquiera (con <code>+</code>)	No	Sí — concatena	No	String (solo Zetariano)
cadena	cualquiera (con <code>-</code> <code>*</code> <code>/</code>)	No	No	No	—
booleano / carácter / clase	cualquiera	No	No	No	—

Tabla 1. Compatibilidad de operandos en operadores aritméticos, por lenguaje.

4.2. Concatenación de cadenas (exclusiva de Zetariano)

En Zetariano, si cualquiera de los dos operandos del operador `+` es de tipo `String`, el resultado de la operación es también de tipo `String`: el otro operando —sea `int`, `double` o `char`, con la salvedad indicada más abajo para `boolean`— se convierte automáticamente a su representación textual. En el código C generado, esta conversión se traduce en una llamada a la función auxiliar `c3d_concat(...)`, que envuelve previamente cualquier operando que no sea ya `char*` con el conversor correspondiente (`c3d_entero_a_cadena`, `c3d_decimal_a_cadena` o `c3d_caracter_a_cadena`). Estas

funciones auxiliares son generadas y emitidas por `GeneradorC` únicamente cuando el programa efectivamente las utiliza, de manera que `gcc -Wall` no reporte advertencias por funciones sin uso.

Existe una limitación conocida, heredada del backend y documentada pero no bloqueante: dado que el código C generado representa tanto `boolean` como `int` mediante el mismo tipo `int`, sin una marca que los distinga, concatenar un valor `boolean` de Zetariano produce actualmente el texto «1» o «0» en lugar de «true» o «false». Esta situación no se ejerce en ninguno de los archivos de ejemplo incluidos en el proyecto.

Y? y Pig Latin no poseen operador de concatenación: declarar, por ejemplo, `cadena a = "x" + "y"`, o combinar una cadena con cualquier otro tipo mediante `+`, constituye un error semántico en ambos lenguajes.

4.3. Comparación de igualdad (`==` y `!=`)

La comparación de igualdad utiliza la regla general de compatibilidad descrita al inicio de esta sección, evaluada en ambos sentidos, y por ello resulta más permisiva que los operadores aritméticos; sin embargo, nunca permite comparar una cadena con un tipo numérico o booleano. Dos cadenas se consideran comparables en los tres lenguajes, y dicha comparación se traduce en C como una llamada a `strcmp` —comparando el contenido de ambas cadenas y no la dirección de memoria de los punteros—. Dos instancias de una misma estructura o clase también son comparables, y la comparación se traduce como una comparación de punteros, es decir, verifica si ambas variables refieren a la misma instancia; en Zetariano, además, una instancia puede compararse contra el literal `null`, lo que se traduce como una comparación de puntero contra `NULL`. No es válido comparar instancias de dos estructuras o clases distintas entre sí.

4.4. Operadores relacionales (`<`, `>`, `<=`, `>=`)

Los operadores relacionales solo aceptan operandos de tipo entero o decimal en los tres lenguajes —a diferencia de la igualdad, no admiten cadenas bajo ninguna circunstancia—, y el resultado de la comparación es siempre de tipo booleano.

4.5. Asignación

En una asignación, el tipo del valor asignado debe ser compatible, según la regla general enunciada al inicio de la sección 4, con el tipo declarado de la variable o del campo destino: mismo tipo exacto, promoción de entero a decimal, o asignación de `null` a un tipo por referencia. No existe conversión implícita entre entero y cadena, y tampoco existe truncamiento implícito de decimal a entero: asignar un valor decimal a una variable declarada como entera constituye un error semántico en los tres lenguajes.

Conclusiones

El diseño descrito en este manual responde a un objetivo central: que los tres lenguajes del proyecto — pese a tener gramáticas, palabras reservadas y estilos sintácticos distintos— compartan un único sistema de tipos, una única tabla de símbolos y un único mecanismo de manejo de errores. Esta decisión de arquitectura es la que permite que Pig Latin actúe como punto de integración de los otros dos lenguajes sin que ninguno de los tres analizadores deba reimplementar la lógica de resolución de tipos o de reporte de errores que ya existe en el núcleo semántico compartido, y es también la que hizo posible detectar y corregir, durante el desarrollo de la interfaz gráfica, un problema de trazabilidad de errores entre archivos importados que no había resultado evidente mientras cada analizador se probaba de forma aislada.